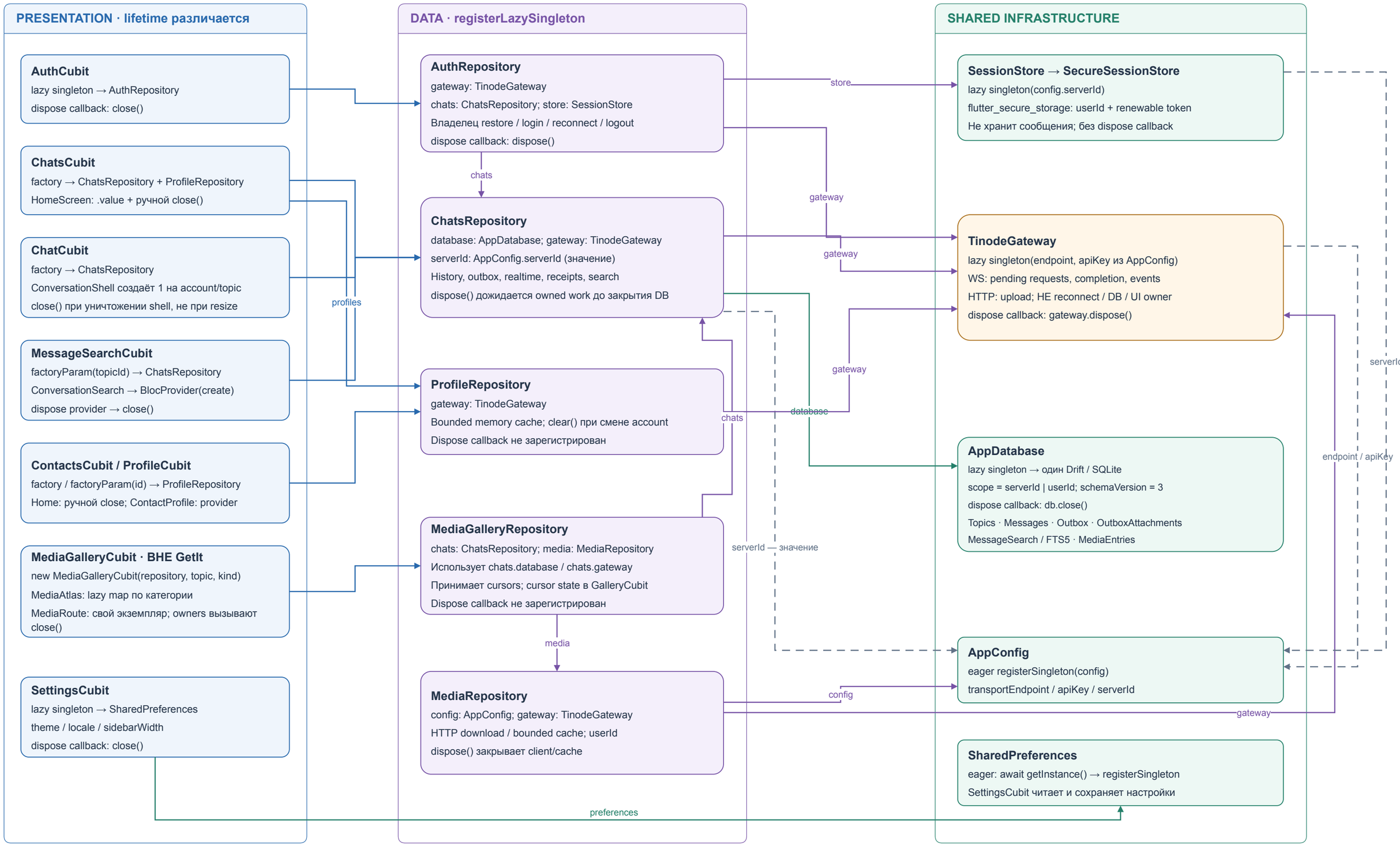
DI: что создаётся, кто получает зависимость и кто закрывает экземпляр

Стрелка A → B означает: A получает B через конструктор. Метки lifetime — реальные регистрации GetIt; Cubit экрана не равен singleton.



Ещё две регистрации в initInjections()

PushService(gateway): lazy singleton + dispose(). StreamSubscription<SessionState>: eager singleton + cancel(). Runtime-связи и фактический startup — вкладка 02.

Источники: lib/dependency_injection.dart; lib/app.dart; presentation/navigation/conversation_shell.dart; media/media_atlas.dart; navigation/media_route.dart · Рабочие исходники, 10.10.2026. Не runtime-трассировка и не новый прогон тестов.

Bootstrap, владение экземплярами и смена сессии

Три разных жизненных цикла: процесс приложения, authenticated account/session и route/pane. Закрытие route не останавливает outbox.

BOOTSTRAP · main() → initInjections() → runApp()

main()
ensureInitialized() / path URL strategy
registerFontLicenses()
await initInjections(AppConfig.environment())

await

initInjections(config)
AppConfig + await SharedPreferences: eager
DB / Gateway / stores / repos: lazy singletons
Cubits: singleton / factory / factoryParam

подписка

Первый locator<AuthRepository>()
changes.listen(...) в конце initInjections
Принудительно создаёт AuthRepository, ChatsRepository, AppDatabase, TinodeGateway и SecureSessionStore
Lazy registration ≠ deferred до login

initInjections завершён

runApp(CollapseApp)
AuthRepository.restore() запущен unawaited
AuthCubit / SettingsCubit — через locator
Репозитории и общие Cubits переданы .value
AppRouter — late final в _CollapseAppState

restore()

AuthRepository.restore()
Нет saved session → signedOut; есть → chats.start → offline → reconnect
Сеть: connect / token login → сохранить новый token → online
refreshTopics / flushOutbox работают асинхронно после local init
Готовый router / online не означает, что вся история синхронизирована

SessionState

changes.listen(SessionState) · синхронные побочные эффекты в composition root
MediaRepository.userId = state.userId → namespace/generation для media cache.
При смене account: ProfileRepository.clear(). При online: unawaited PushService.synchronize().
Эта подписка хранится в GetIt; dispose callback только subscription.cancel().

ВЛАДЕЛЬЦЫ / DISPOSE · НЕ СМЕШИВАТЬ

_CollapseAppState → AppRouter(AuthCubit, AppConfig)
AuthCubit.stream → notifyListeners → GoRouter.redirect; отдельная подписка.
app.dispose(): Router subscription.cancel + GoRouter.dispose; .value не закрывает shared.
В lib нет locator.reset(): закрытие app не является reset всего DI-графа.

ConversationShell → ChatCubit + ChatScreen
Cubit создаётся один раз в initState, key определяется account/topic.
Info/search и resize меняют pane geometry; Cubit, draft и window остаются.
shell.dispose() закрывает ChatCubit; outbox остаётся в ChatsRepository.
MediaRoute и MediaAtlas сами создают/закрывают MediaGalleryCubit.

Явный await GetIt.reset(): callbacks в обратном порядке регистрации
subscription → AuthCubit → SettingsCubit → AuthRepository → PushService
→ MediaRepository → ChatsRepository → TinodeGateway → AppDatabase.
ChatsRepository.dispose ждёт owned work + Drift barrier перед DB close.
AuthRepository.dispose не делает logout и не ждёт _operation.

SESSION · AuthRepository управляет политикой; TinodeGateway — только транспортом

Disconnect / reconnect
Gateway.events(disconnected) → AuthRepository
saved session остаётся → emit offline
retry/backoff → connect + token login
Тот же account продолжает durable outbox

явный выход

UI: await PushService.logout() → AuthCubit.signOut()
AuthRepository: ++_epoch; отменить retry; очистить _saved
emit signedOut сразу; disconnect socket
дождаться предыдущей auth operation → disconnect ещё раз
await chats.stop() → await store.clear()
signIn ждёт _signingOut barrier; кеш сообщений остаётся

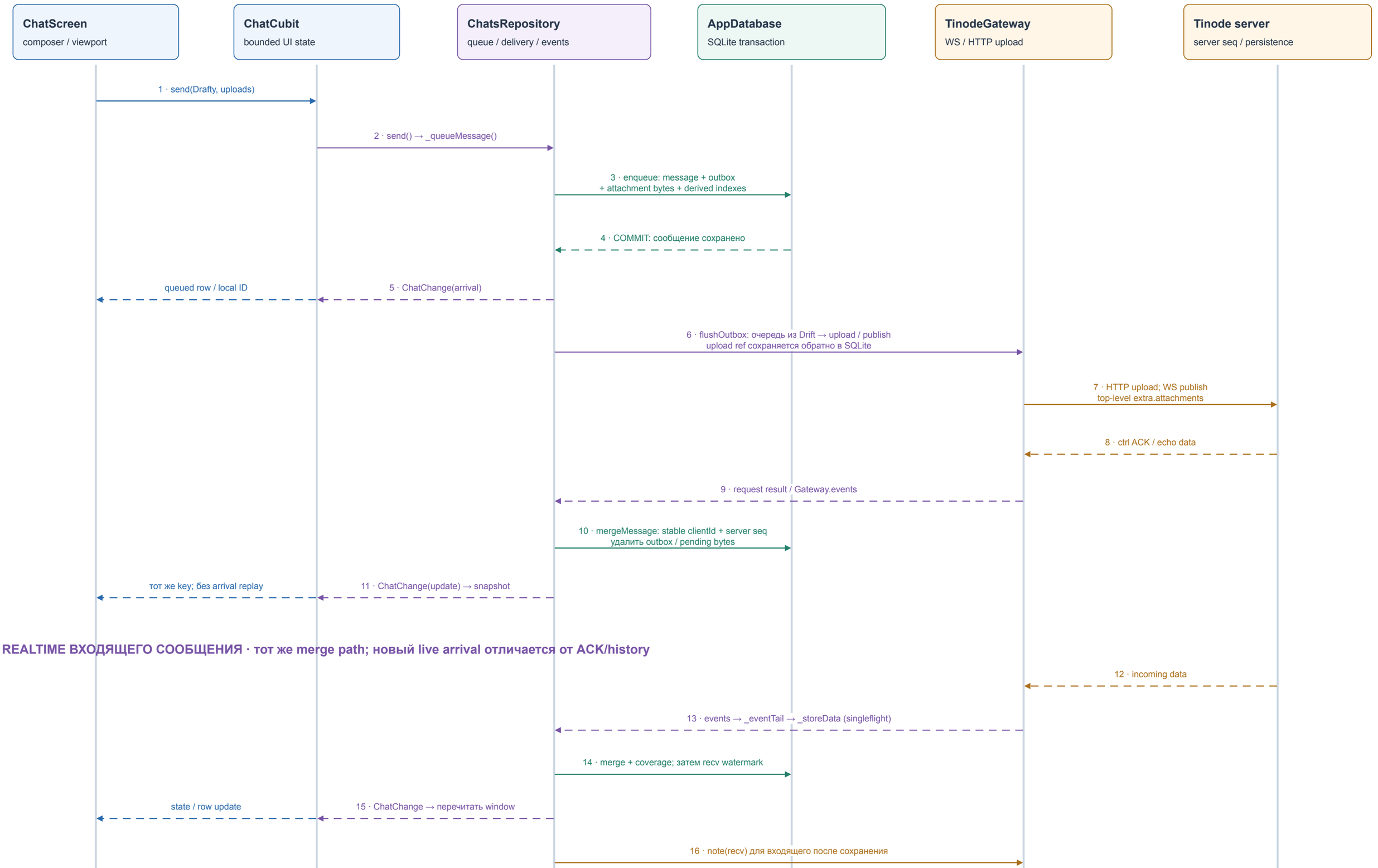
граница сессии

Защита от старой async-работы
AuthRepository: epoch; ChatsRepository: scope + session generation.
Late accepted ACK может слиться в исходный account cache.
Старый worker не должен публиковать новые сообщения за новую сессию.
Пауза pane/app влияет на read/playback, а не на уже сохранённый outbox.
GetIt instances общие; account isolation реализован внутри владельцев.

Источники: main.dart; app.dart; dependency_injection.dart; data/auth/{auth_repository,auth_restore,auth_session,auth_reconnect}.dart; data/chat/session_lifecycle.dart · Рабочие исходники, 10.10.2026. Не runtime-трассировка и не новый прогон тестов.

Сообщение: durable enqueue → network delivery → ACK/echo merge

send Future означает durable enqueue. После COMMIT: UI и unawaited(flushOutbox) независимы. Paint/mount не условие доставки. Lost ACK ≠ exactly-once.



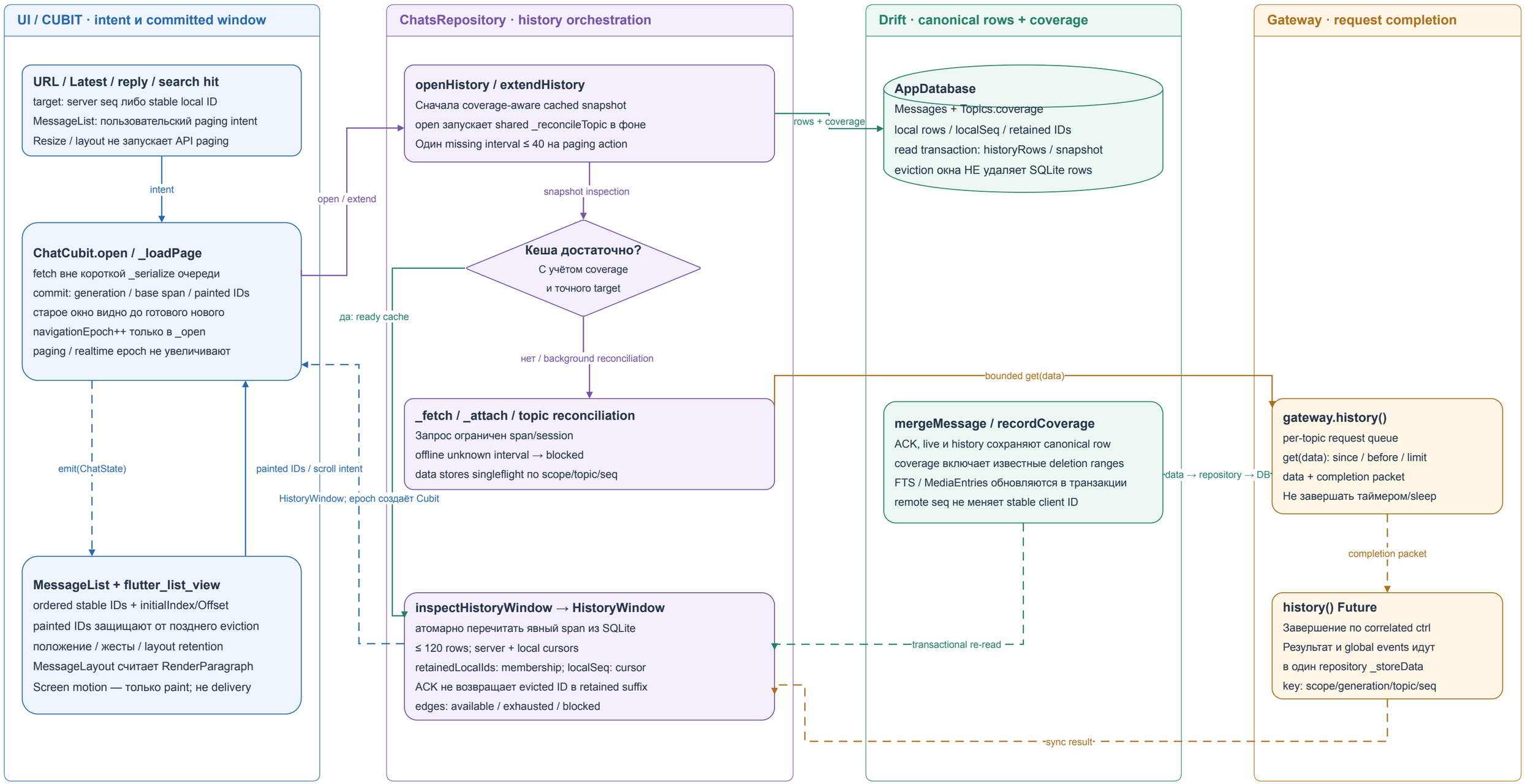
Read — отдельный поток

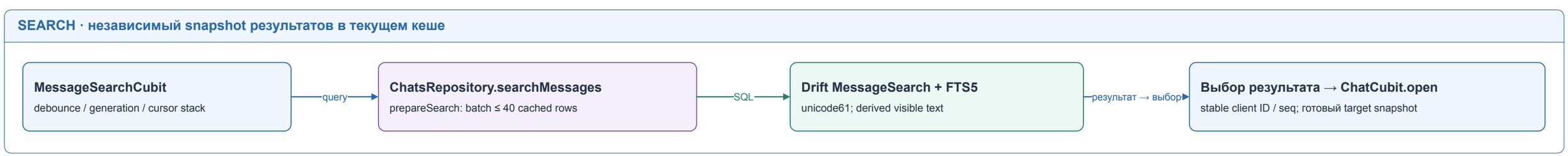
Видимый incoming + active app/route/pane → ChatCubit.markVisible → ChatsRepository.markRead → DB read watermark → gateway.note(read). ACK сам по себе не читает чат.

Источники: data/chat/{message_queue,outbox_delivery,realtime_events,message_actions}.dart; database/{outbox_queries,message_merge}.dart; presentation/chat/chat_receipts.dart · Рабочие исходники, 10.10.2026. Не runtime-трассировка и не новый прогон тестов.

История: I/O, coverage, bounded window и viewport — разные владельцы

В каждой зоне указаны реальные entry points. Окно не является базой данных; scroll position не является сетевым курсором.





_reconcileTopic в ChatsRepository эмитит HistorySyncPhase.started/completed/failed; Cubit заменяет кешированный хвост только при живом follow intent.
loadMoreSearchHistory явно получает один missing interval ≤ 40. Query — только SQLite; явная загрузка может дать recv, но не displayed read.

Источники: presentation/chat/{chat_open,chat_paging,message_list*,chat_live}.dart; data/chat/{window_loading,history_snapshot,message_search,search_navigation}.dart · Рабочие исходники, 10.10.2026. Не runtime-трассировка и не новый прогон тестов.

Media: два независимых графа — metadata selection и original bytes

Галерея использует общий ChatsRepository для scope/attach/DB/Gateway, но не владеет ChatCubit, history window или viewport чата.

A · METADATA · get.what=media → meta.media

MediaAtlas / MediaRoute
Создают свой MediaGalleryCubit
category + cursors + snapshot
selected stable media ID

open / refresh
older / newer

MediaPage
Cubit.emit → UI

MediaGalleryRepository.page / cached
account / generation из ChatsRepository
prepareMediaTopic(): attach + sync deletions
cached → Drift.mediaPage; online → gateway.media
После ответа cacheMedia(check: generation)
403/404 → clearMedia; original entity key сохранён

cached / cacheMedia

Drift · MediaEntries
Metadata only; schema 3
(scope, topic, seq, entityIndex)
clientId стабилен pending → ACK
Локальные записи индексируются batch
Не копия Messages / original bytes

Gateway.media → Tinode overlay
get.what=media: kind / cursor / snapshot
attached-topic Read ACL
PostgreSQL по (seq, original entity key)
meta.media: коррелированная страница
deletions / access loss учитываются

gateway.media()

meta.media → validate → cache

Граница metadata path
meta.media не попадает в обычные data-events, ChatCubit, history coverage, read или recv. Offline atlas явно частичный; metadata cache не означает наличие оригинала.

B · BYTES · MediaGalleryCubit.bytes → MediaGalleryRepository.bytes(item)

bytes(item)
capture scope + generation
check после async I/O
Источник: pending / inline / ref

pending

Pending → ChatsRepository.pendingAttachmentBytes
SQLite OutboxAttachments; локальные байты до upload
Отдельная ветка, не HTTP-кеш MediaRepository

return Uint8List

Viewer / save получают Uint8List
Pending → байты напрямую; image / save обходят MediaRepository cache
Audio/video могут вызвать MediaRepository.playableBytes
Если pending уже ACK: fallback к актуальной Drafty entity

Uint8List → caller

inline без оригинала

Inline без ref/val → ChatsRepository.mediaMessage
Сначала Messages; если нет — _fetch ровно seq..seq+1, limit=1
Это обычный data path: сохраняет coverage и возможный recv
read не посылает; промежуточную историю не загружает

original Drafty entity

ref / val

Ref / inline val → MediaRepository
draftyReference → bytes / playable
Trust origin + auth headers; bounded HTTP and cache

MediaRepository.bytes()

MediaRepository: ref → HTTP/cache; val → inline decode
Native: cached first + stale revalidation; Web: GET on miss
Web RAM: 64 MiB / 200 entries; reload не сохраняет originals
Key: serverId | userId | ref; HTTP redirects disabled
data URI → _inlineBytes напрямую; auth только trusted origin

auth GET ↔ bytes / cache

HTTP files: native → Tinode; web → Caddy /v0/
bytes → viewer; active route/app/pane управляют playback pause

Сохранение идентичности / invalidation
entityIndex не переиндексируется; pending → ACK сохраняет media identity. Gallery changes слушают chats.changesFor(topic); stale session response не должен попасть в новый cache.

Источники: data/media/{media_gallery_repository,media_repository,web_media_cache}.dart; data/chat/chats_repository.dart; presentation/media/media_gallery_cubit.dart; backend/MEDIA_API.md · Рабочие исходники, 10.10.2026. Не runtime-трассировка и не новый прогон тестов.

DI reference: все 19 регистраций и прямые constructor dependencies

Источник — lib/dependency_injection.dart. S: eager singleton; L: lazy singleton; F: factory; FP: factoryParam. Callback относится к DI, не к logout.

Тип / registration key	Lifetime	Constructor arguments	Dispose callback / owner	dependency_injection.dart
AppConfig	S	переданный config	—	строка 25
SharedPreferences	S	await SharedPreferences.getInstance()	—	строка 26
AppDatabase	L	internal driftDatabase(...)	db.close()	строка 27
TinodeGateway	L	config.transportEndpoint, config.apiKey	gateway.dispose()	строка 31
SessionStore	L	SecureSessionStore(config.serverId)	—	строка 38
ChatsRepository	L	AppDatabase, TinodeGateway, config.serverId	repo.dispose()	строка 41
ProfileRepository	L	TinodeGateway	—	строка 49
MediaRepository	L	AppConfig, TinodeGateway	repo.dispose()	строка 50
MediaGalleryRepository	L	ChatsRepository, MediaRepository	—	строка 54
PushService	L	TinodeGateway	service.dispose()	строка 57
AuthRepository	L	TinodeGateway, ChatsRepository, SessionStore	repo.dispose()	строка 61
SettingsCubit	L	SharedPreferences	cubit.close()	строка 66
AuthCubit	L	AuthRepository	cubit.close()	строка 70
ChatsCubit	F	ChatsRepository, ProfileRepository	владелец factory instance	строка 74
ContactsCubit	F	ProfileRepository	владелец factory instance	строка 75
ChatCubit	F	ChatsRepository	ConversationShell.dispose()	строка 76
MessageSearchCubit	FP	ChatsRepository, topicId (param1)	BlocProvider(create)	строка 77
ProfileCubit	FP	ProfileRepository, userId (param1)	Home / ContactProfile	строка 80
StreamSubscription<SessionState>	S	AuthRepository.changes.listen(...)	subscription.cancel()	строка 96

Создаются напрямую, вне GetIt

AppRouter(AuthCubit, AppConfig): _CollapseAppState; app.dart:24.

MediaGalleryCubit: MediaRoute + MediaAtlas; media_route.dart:30, media_atlas.dart:20.

Invite ContactsCubit: ProfileScreen dialog provider; profile_invitation.dart:4.

HTTP/cache/secure storage — внутренние зависимости соответствующих owners.

Границы, важные при изменении архитектуры

Widgets → Cubit → repository — основной, но не единственный маршрут вызова.

ChatScreen также получает repos для draft/profile/media/actions; SQL/socket остаются в data.

Home создаёт 3 Cubit и закрывает вручную; .value ничего не закрывает.

Scope меняется внутри общих instances; logout не создаёт новую SQLite/DI-контейнер.

Источники: dependency_injection.dart:24–100; app.dart:24–55; home_screen.dart:52,93,137; route_pages.dart:67,131; conversation_shell.dart:53,170 · Рабочие исходники, 10.10.2026. Не runtime-трассировка и не новый прогон тестов.